Module 5

Problem statement A

Build an ML model that can generate synthetic spoken digits (0–9) after learning from a **Spoken Digit Dataset (FSDD)**—a collection of short audio clips of people speaking digits in English.

Your model should be able to:

Generate new unheard spoken digits

Dataset Description (FSDD)

- **Dataset:** Free Spoken Digit Dataset (FSDD)
- **Data type:** WAV files (recorded speech)
- **Classes:** Spoken digits (0–9)
- **Speakers:** Multiple (different voices)
- **Sampling rate:** 8 kHz

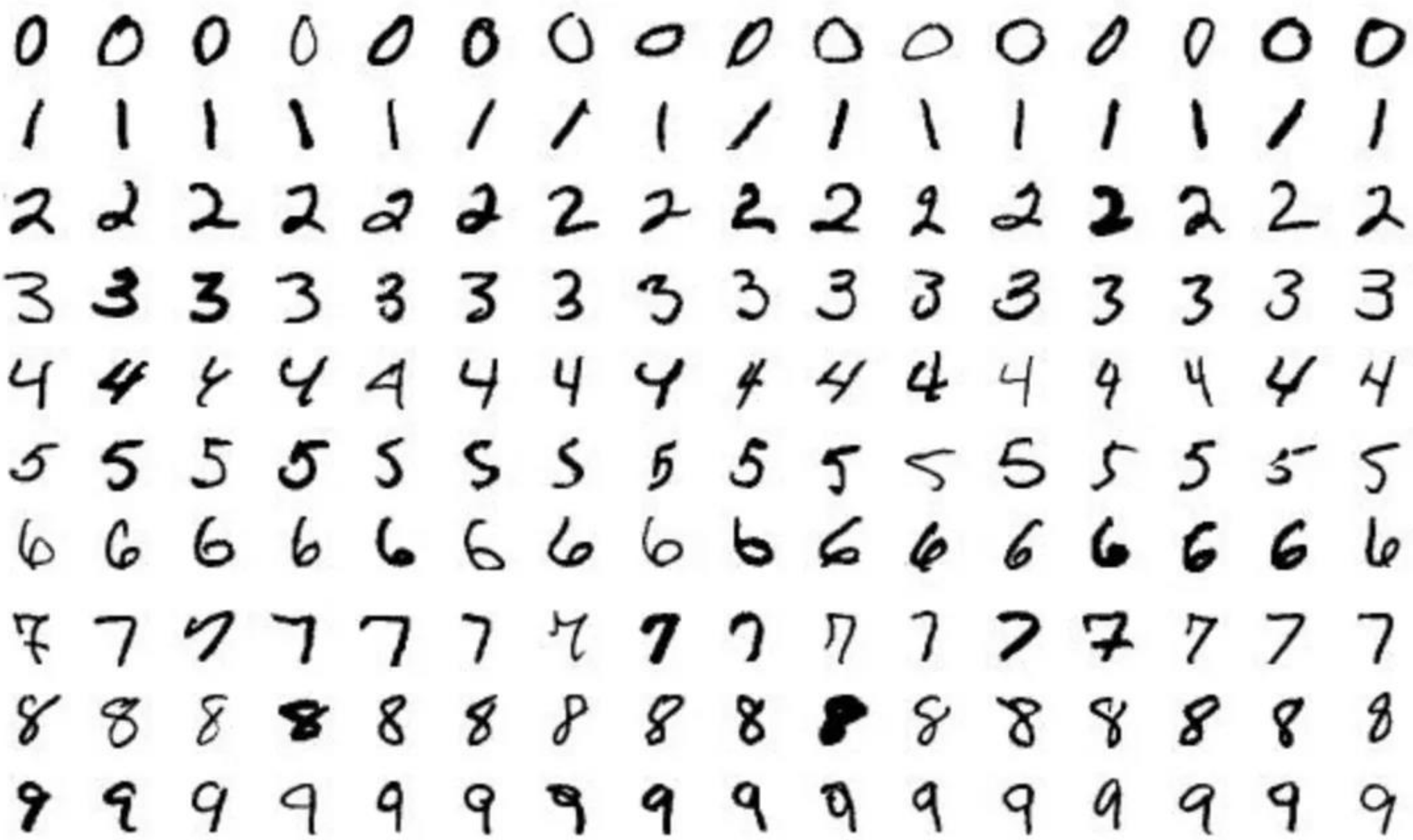
Problem statement B

Build an ML model that can generate synthetic handwritten images of digits (0–9) after learning from MNIST dataset—a collection of short audio clips of people speaking digits in English.

Your model should be able to:

Generate new unseen images (which are not present in MNIST)

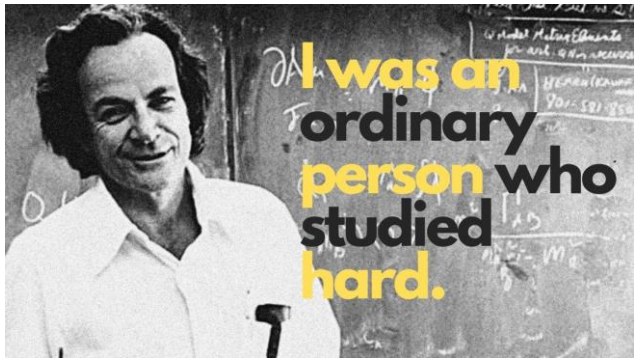
MNIST Dataset (Modified National Institute of Standards and Technology)



We want a model with creativity

Generative Models

Concept	Analogy / Explanation
Discriminative model	Learns *boundaries* between classes (e.g., “this sound is 3, not 5”).
Generative model	Learns *how each class looks or sounds like* so it can *create new examples*.



Feynman's Quote

"What I cannot create, I do not understand."

— *Richard P. Feynman (1918–1988)*

In physics, it meant: "If I can't build the system from scratch, I haven't really grasped how it works."

Context	Meaning of "create"	Meaning of "understand"
Discriminative ML	Model *labels* or *classifies* existing data	Partial understanding — knows differences
Generative ML	Model *creates* new realistic data	Deeper understanding — knows *how* it's made

Discriminative models *see* data and decide.

Generative models *understand* data and imagine.

Probability distributions

We have:

- Input data: x (e.g., audio features of spoken digits)
- Label: y (e.g., the digit spoken: 0–9)

We ultimately want to make predictions:

$$\text{Given } x, \text{ predict } y.$$

Discriminative Model → Learns $P(y|x)$

$$P(y|x) = \frac{P(x,y)}{P(x)}$$

Since we only care about classifying y for a given x, we **don't need to model how x was generated.**

Generative Model → Learns $P(x, y)$ or $P(x|y)$

A generative model tries to **capture the process of how data is produced**.

$$P(x, y) = P(x|y)P(y)$$

or equivalently,

$$P(x) = \sum_{y} P(x|y)P(y)$$

Example (Spoken Digit)

- Learns the probability distribution of features for digit "3" → $P(x|y = 3)$
- Then can *generate* new "3" sounds by sampling from that distribution.

Can we use generative models for Classification?

Explain mathematically

How to develop Generative models?

- We need *feature representations* in generative models
- We have already seen how to learn features in discriminative models (like CNNs).

Can’t we use the same method?

Why Representation Learning is different for generative models

Both learn *representations*, but their **goals differ**.

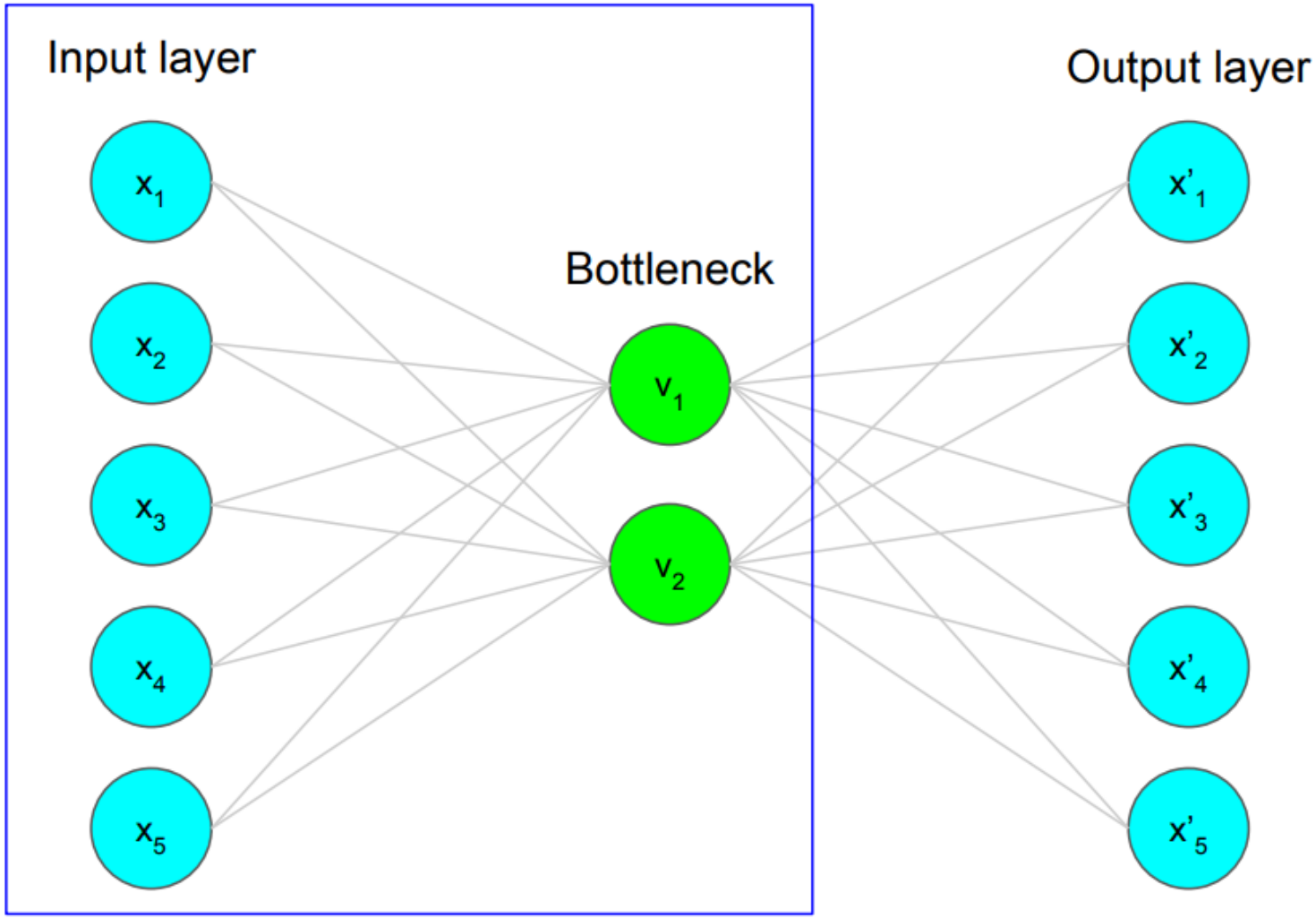
- In **discriminative models**, representations are *class-separating*.
- In **generative models**, representations are *data-descriptive* — they encode *how* data is distributed.

Latent space (Feature space)

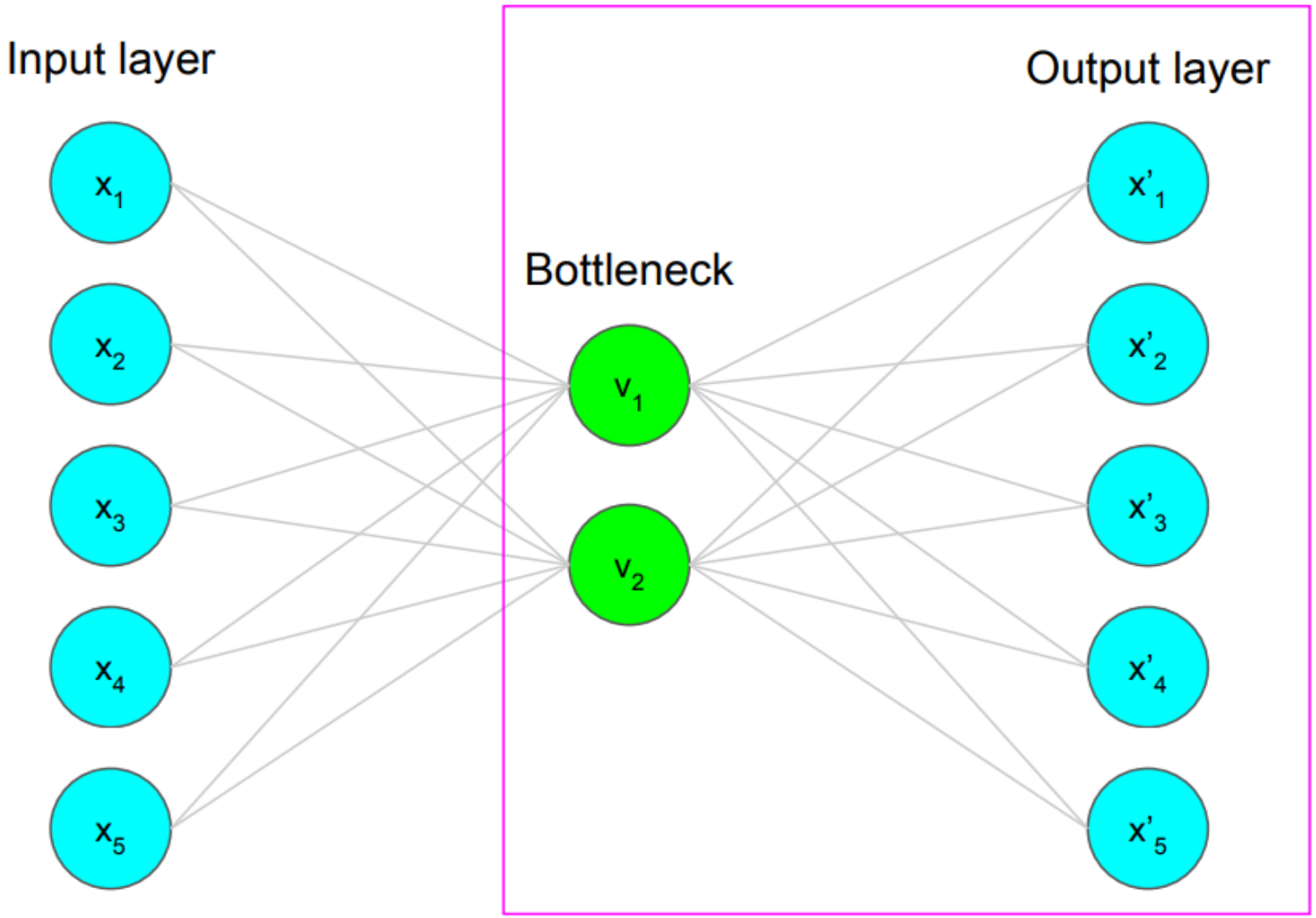
```
Discriminative Model:
    x ⟶ Encoder ⟶ Feature h ⟶ Classifier ⟶ y
                  (class-separating features)

Generative Model (VAE):
    x ⟶ Encoder ⟶ Latent z ⟶ Decoder ⟶ x̂
                  (data-reconstructive features)
```

Compression: Real-world data (e.g., audio waveforms) is high-dimensional. Latent variables z allow the model to capture only the *essential* structure.



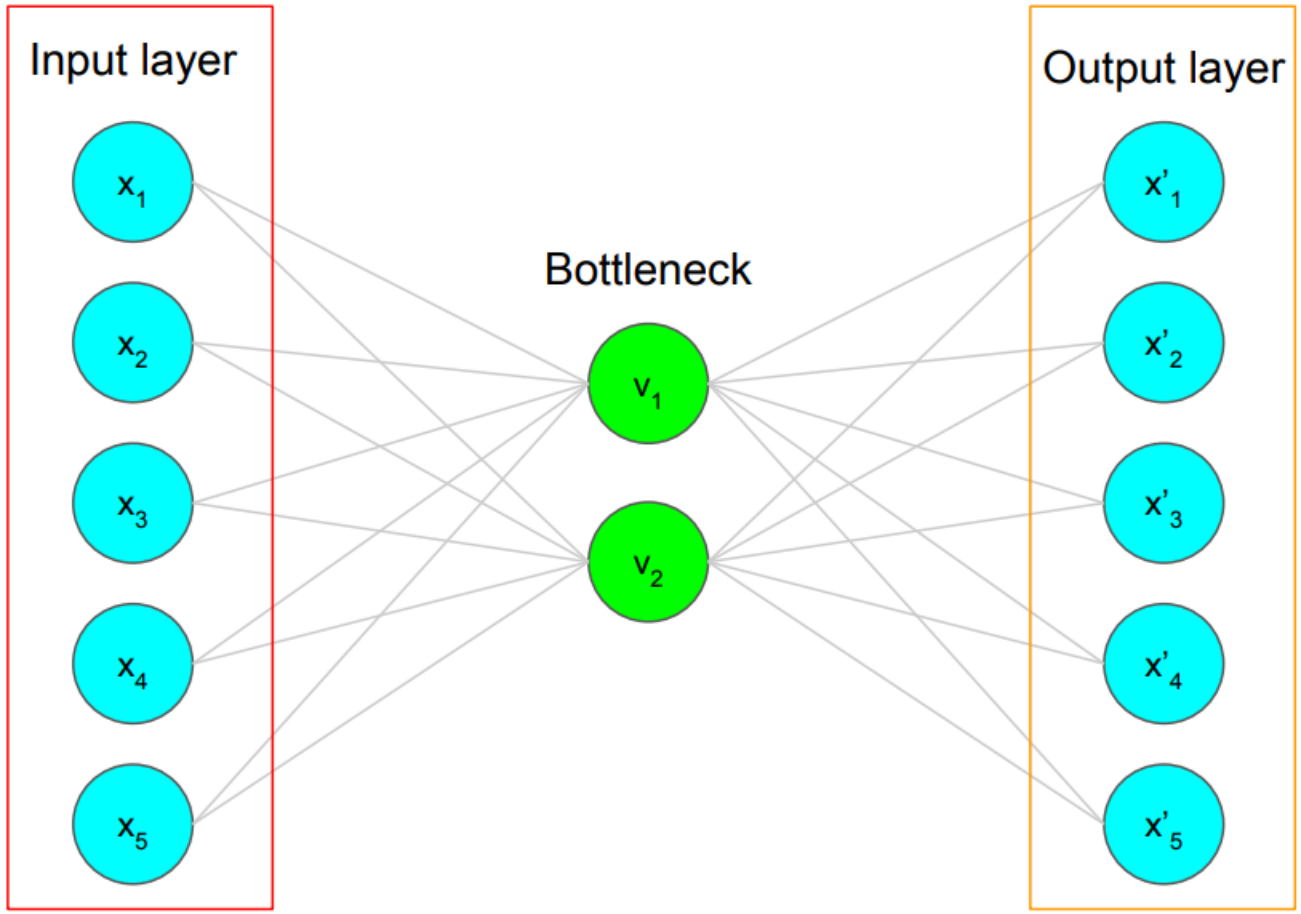
Encoder = compress data into lower-dimensional representation (*latent* space)



Decoder = Decompress representation back to original domain

Autoencoder (AE)

latent space or latent representation)



Original data

Reconstruction

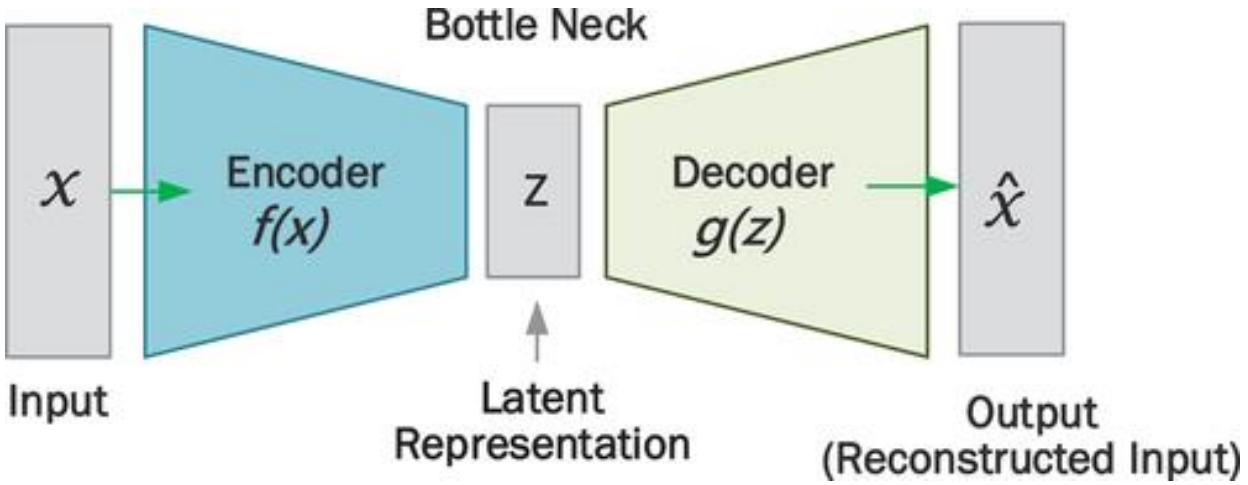
It's the narrowest point in the network — where the model is forced to represent only the most essential features of the data.

$$x \xrightarrow{\text{Encoder}} z \xrightarrow{\text{Decoder}} \hat{x}$$

Why is it called Bottleneck?

Just as a bottleneck restricts flow, the latent layer restricts **information capacity**.

So the model must **compress important patterns** and discard unnecessary noise.



Mathematically

$$z = f_\theta(x) \qquad (\text{Encoder})$$
$$\hat{x} = g_\phi(z) \qquad (\text{Decoder})$$

where:

- f_θ → mapping from input to latent code (parameters θ)
- g_ϕ → mapping from latent code to reconstruction (parameters φ)

Unsupervised learning

In an **Autoencoder**, there are **no external labels**.

The model learns only from the **input itself**.

$$E(x, \hat{x})$$

Original data Reconstructed data

Training pairs look like this:

$$(x, \hat{x})$$

- The **input** x: is, say, the audio spectrogram of a spoken "3".
- The **target output** $\hat{x}$: is also the *same* spectrogram — the model tries to reproduce it.

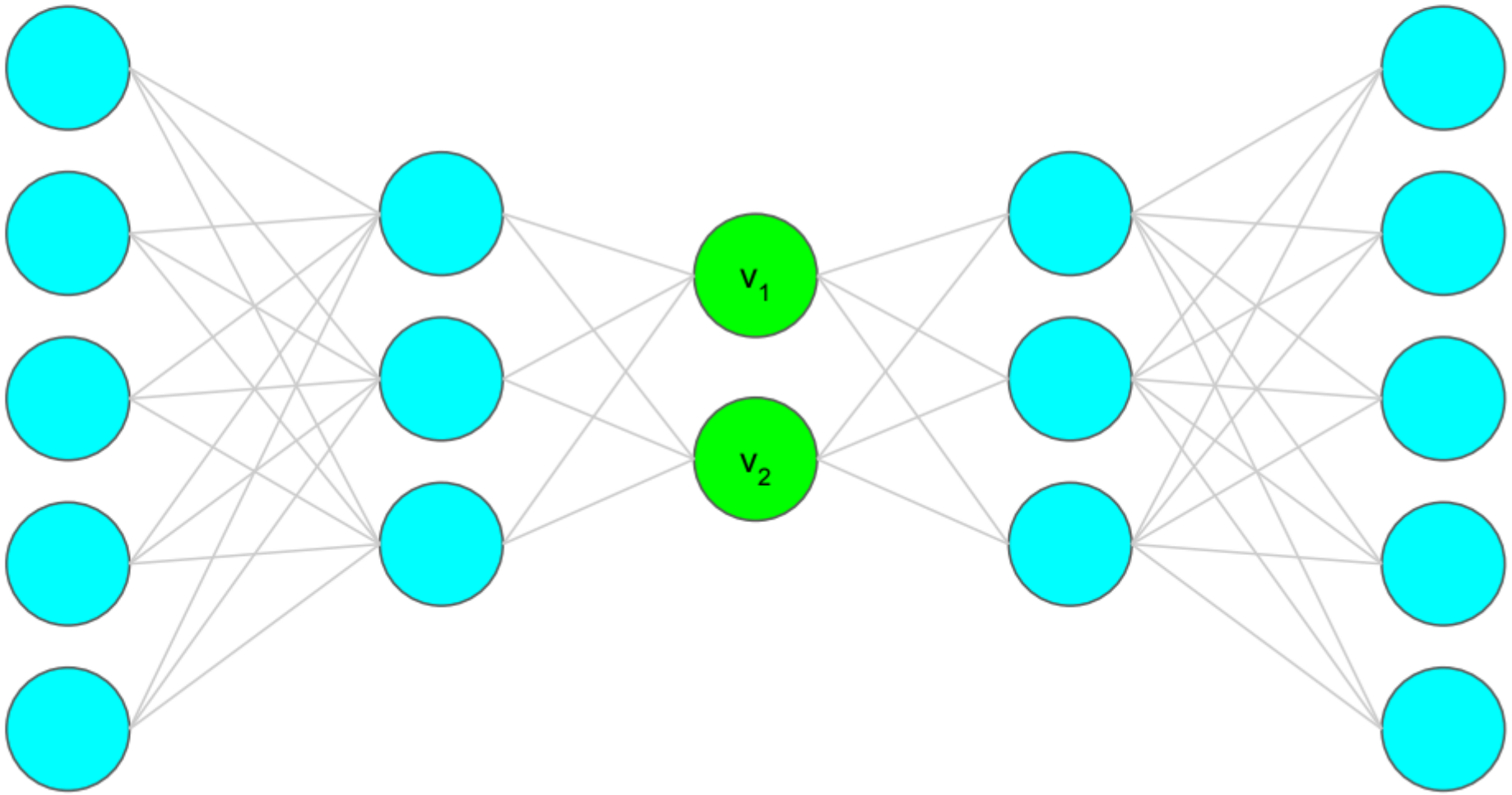
So the model learns to:

$$f(x) \approx x$$

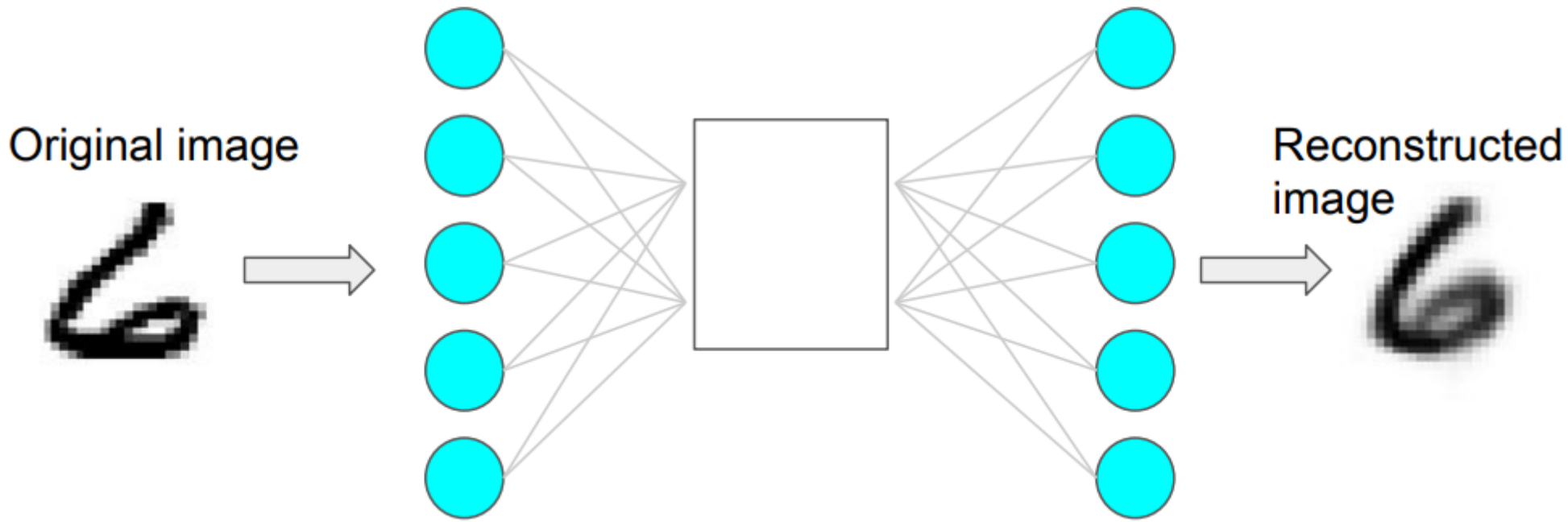
That's why it's **unsupervised learning** —

no label is provided; the model's *goal is to understand and reconstruct its own input.*

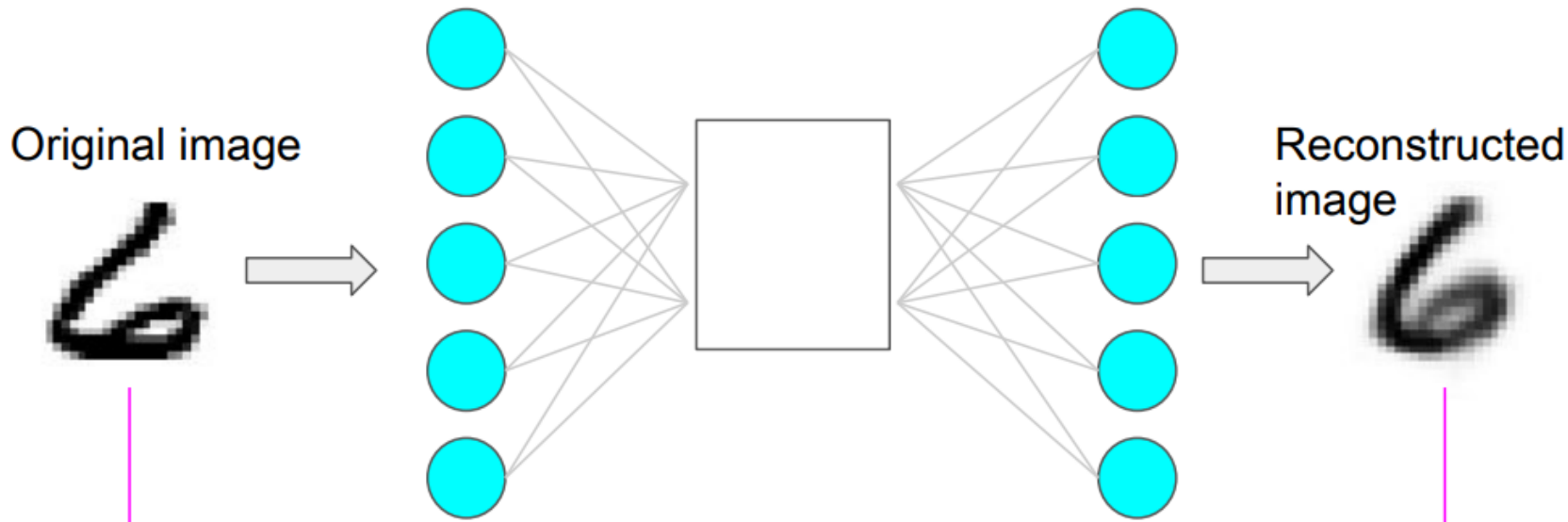
Deep Autoencoder



Loss function: Autoencoder



Loss function: Autoencoder



Training goal: Minimise difference between original and reconstructed image

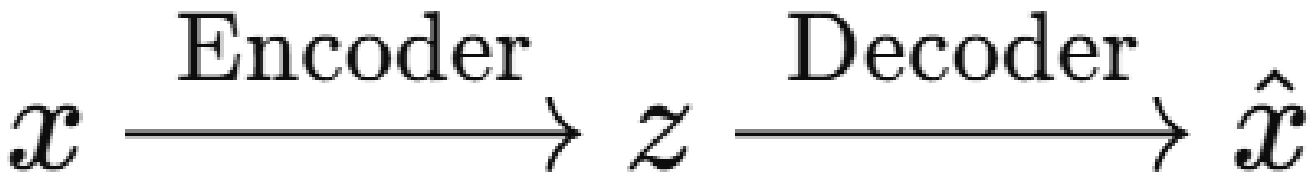
$$LOSS = RMSE$$

Training minimizes the **reconstruction loss**:

$$L = ||x - \hat{x}||^2$$

so that the output looks like the input.

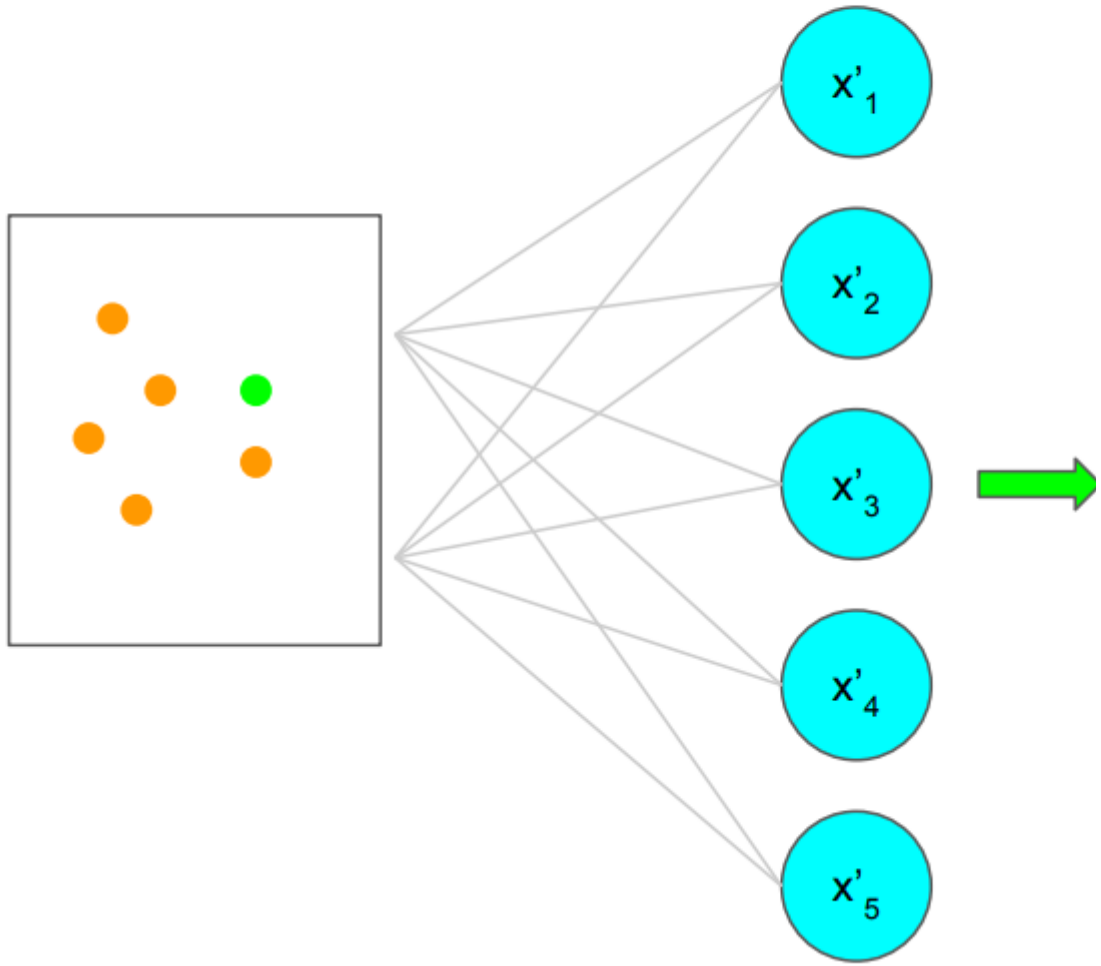
The latent space keeps the most important attributes of the input data

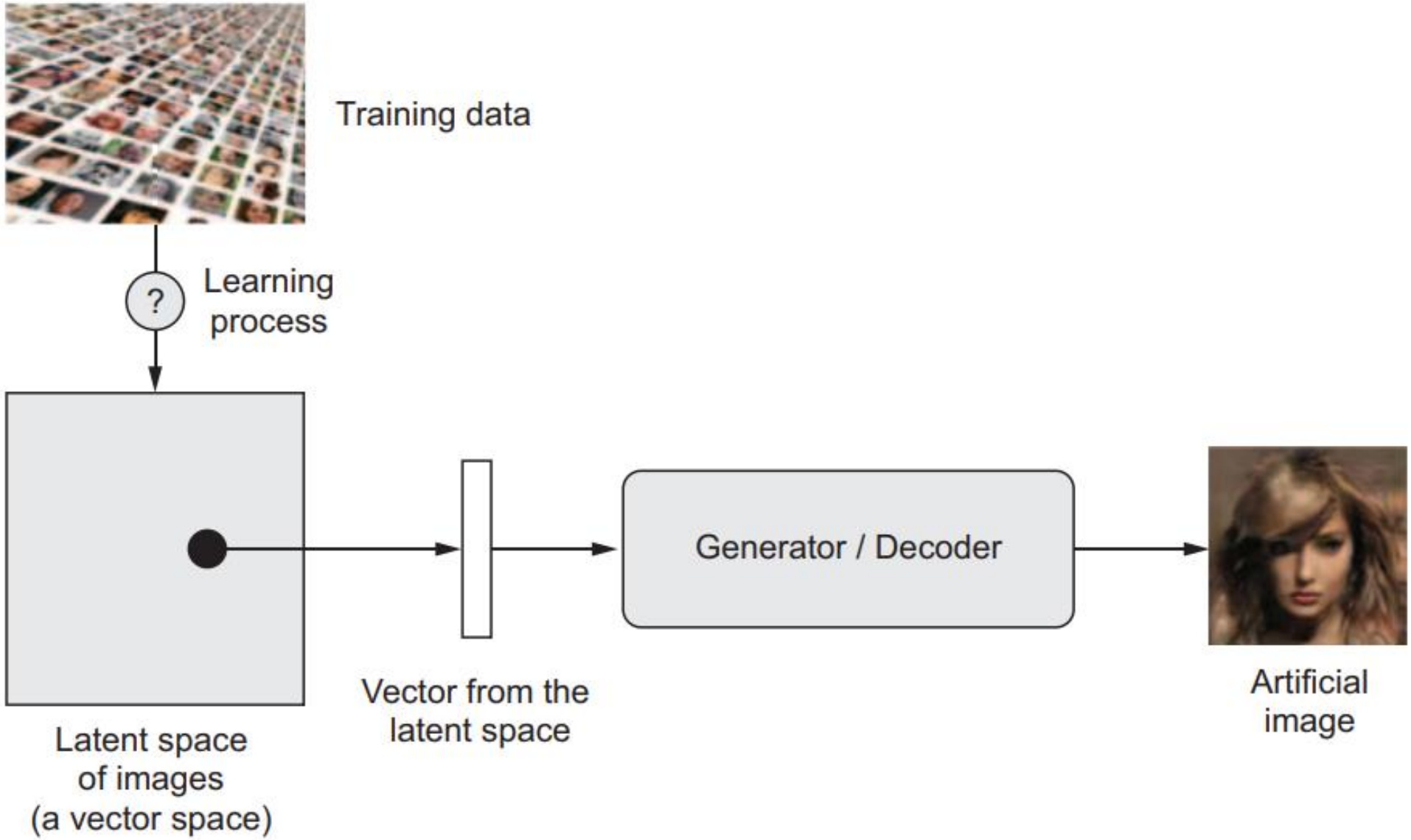


We can leverage the latent space to perform several interesting tasks

Generation with AEs

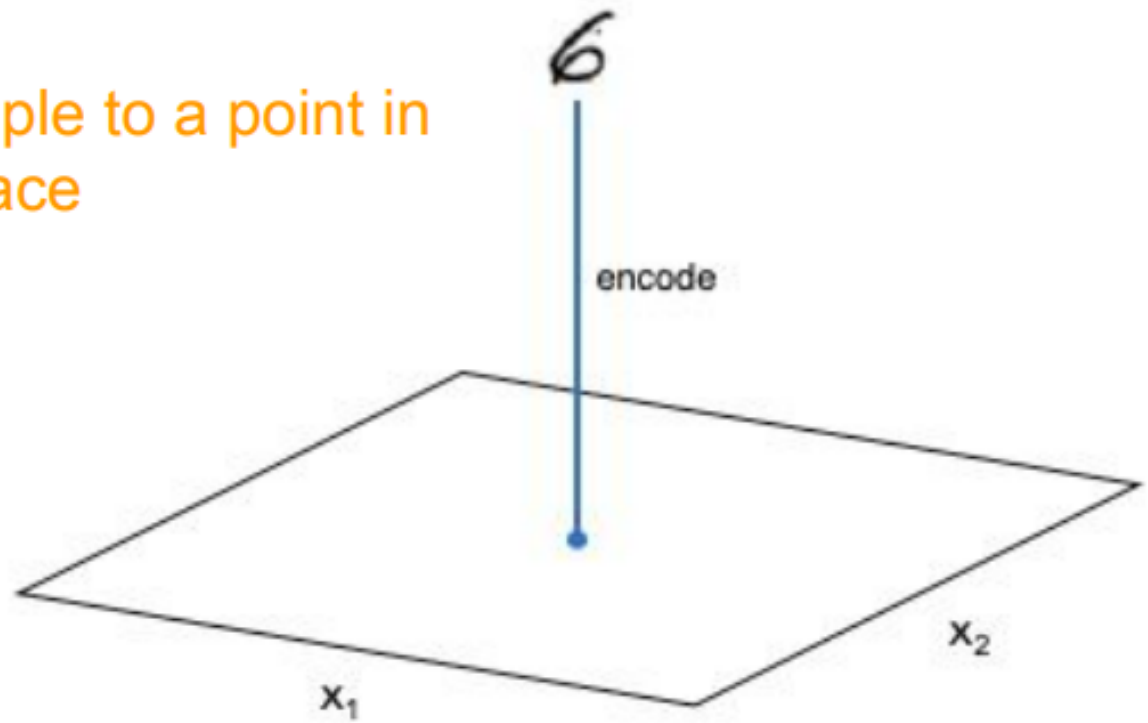
Sample a point in the latent space and pass it through the decoder



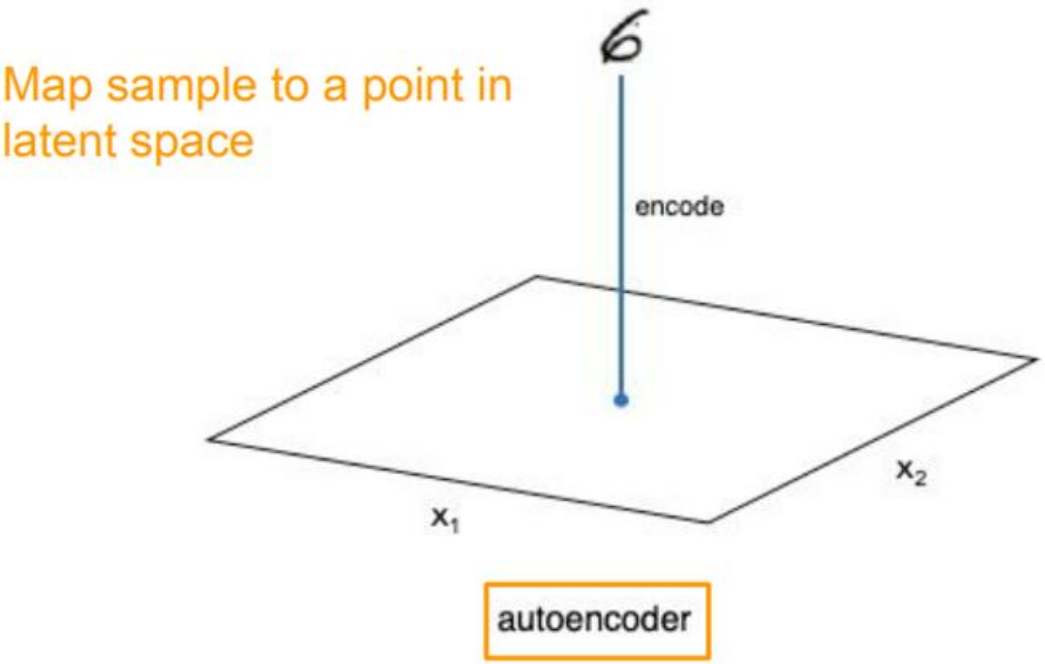

Training data
?
Learning process
Latent space of images (a vector space)
Vector from the latent space
Generator / Decoder
Artificial image

Encoder mapping: AE

Map sample to a point in latent space



autoencoder



(a) Latent space is discontinuous or unstructured

- The AE's latent space z has **no regular pattern** — nearby points in z-space might produce totally different outputs.
- You can't *sample* new z values and expect them to yield valid data.

So:

If you pick a random z (e.g., $[0.2, 0.5, -0.1]$),

the decoder might output *nonsense noise* — not a valid spoken digit.

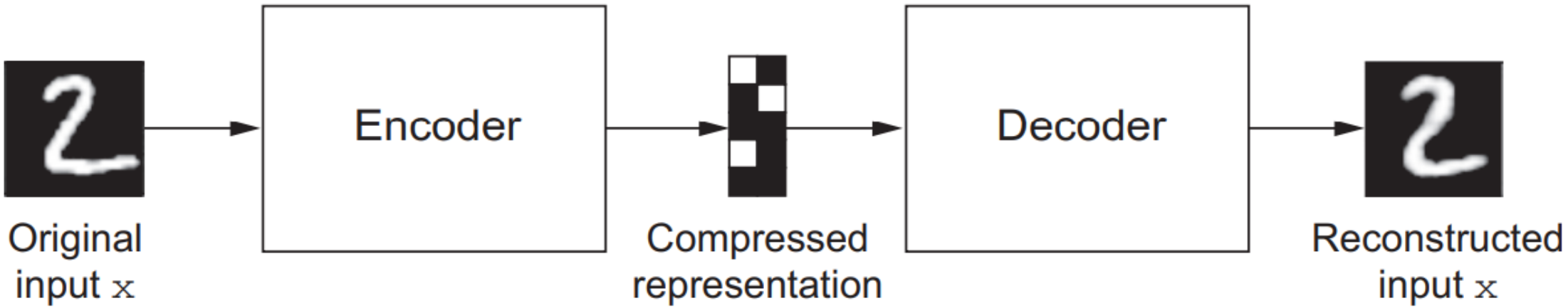
(b) AE has no control over distribution

- The encoder may map training examples to arbitrary clusters in latent space.
- There's no constraint like "make the latent codes roughly Gaussian or smooth."

So the model memorizes how to **rebuild inputs**,

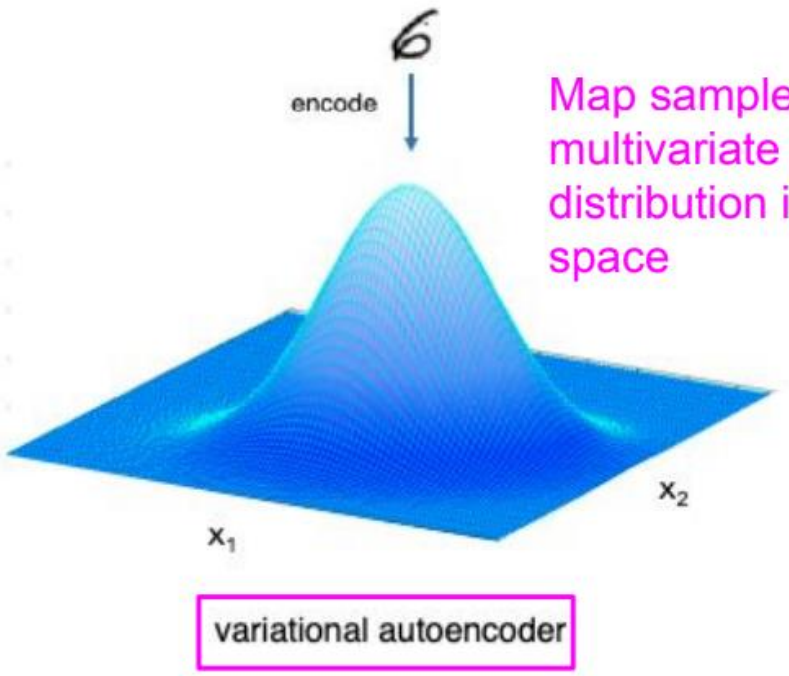
but doesn't learn the *probability distribution* of data — which is required for **generation**.

Creativity of Auto Encoder is Limited



AE are capable of learning dense representations of the input data, called latent representations or codings, without any supervision (i.e., the training set is unlabeled). These codings typically have a much lower dimensionality than the input data, making autoencoders useful for dimensionality reduction

Variational Autoencoder



VAEs address these issues by making the latent representation probabilistic rather than deterministic.

Encoder output:

Instead of a single vector z,

the encoder outputs **mean (μ)** and **variance (σ)** — defining a *distribution*:

$$q(z|x) = \mathcal{N}(\mu(x), \sigma^2(x))$$

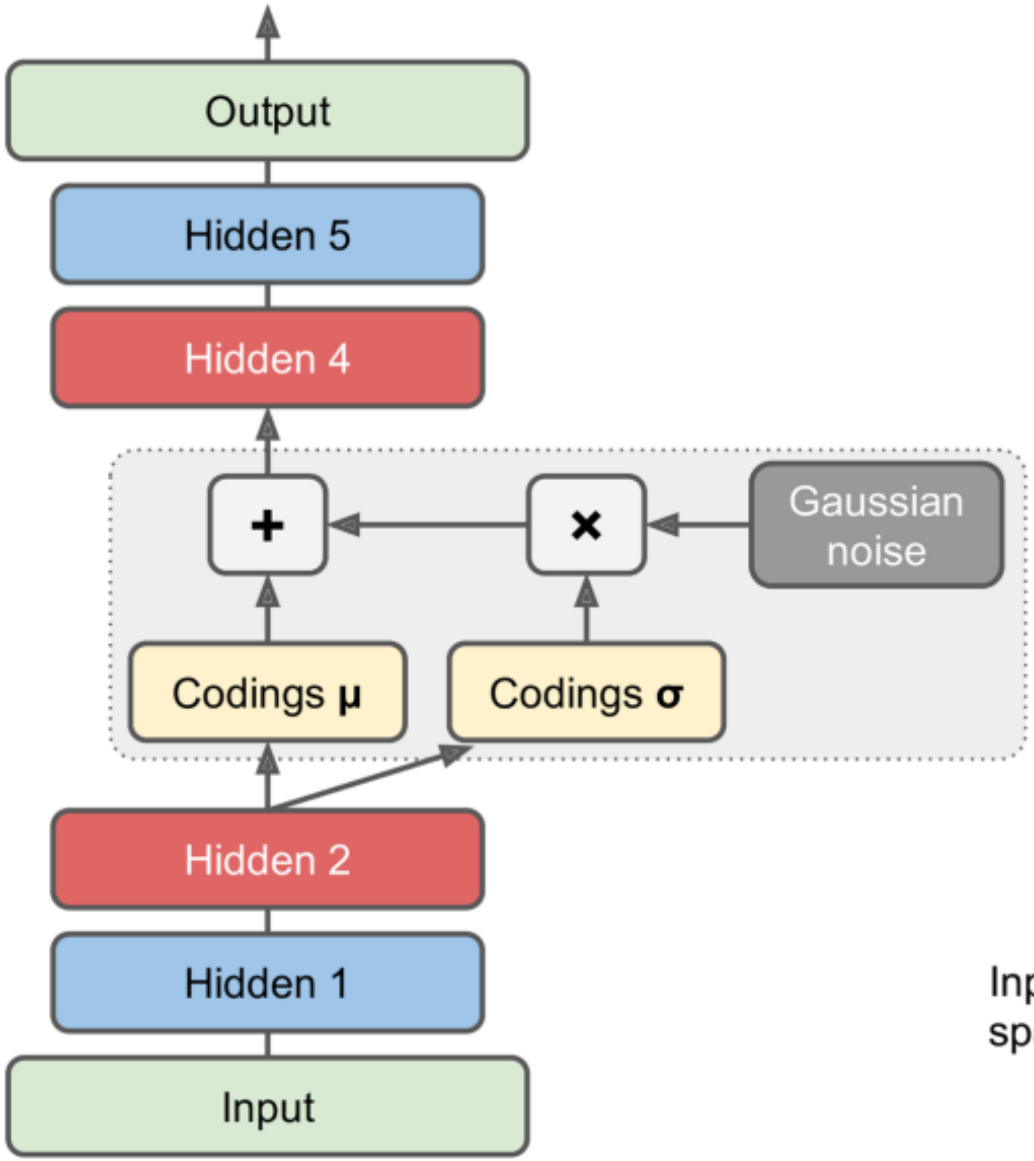
Then, during training:

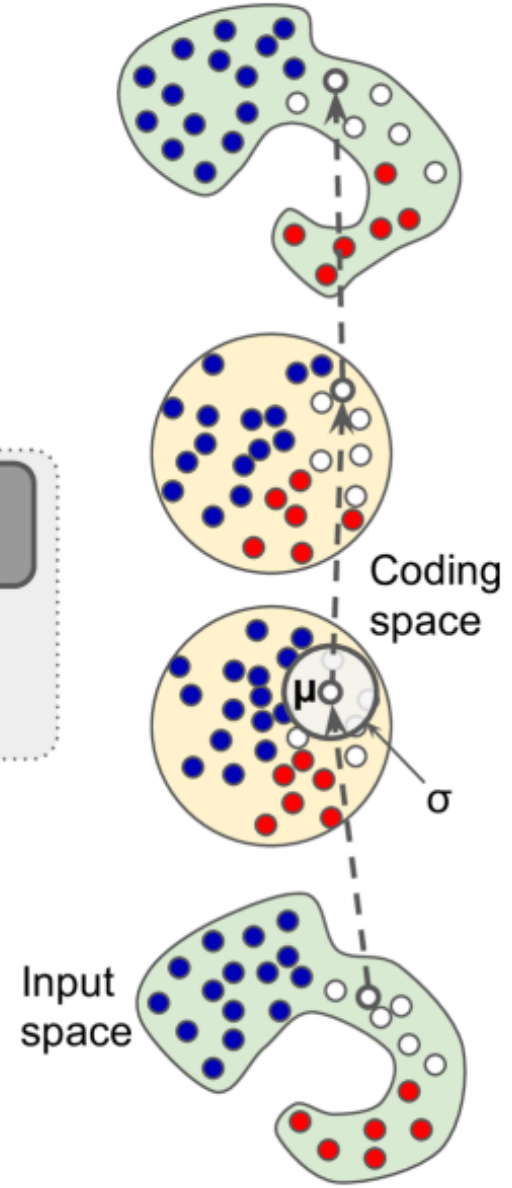
$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

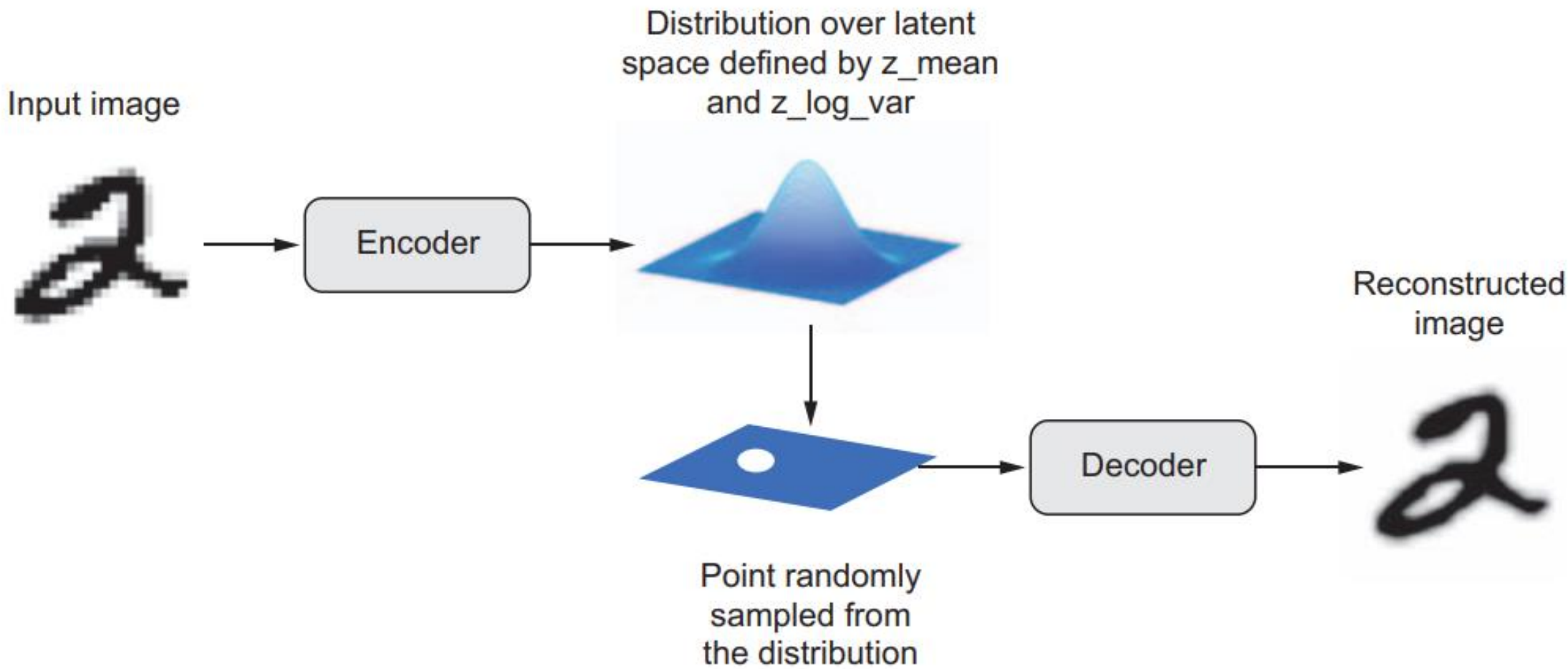
Variational autoencoder

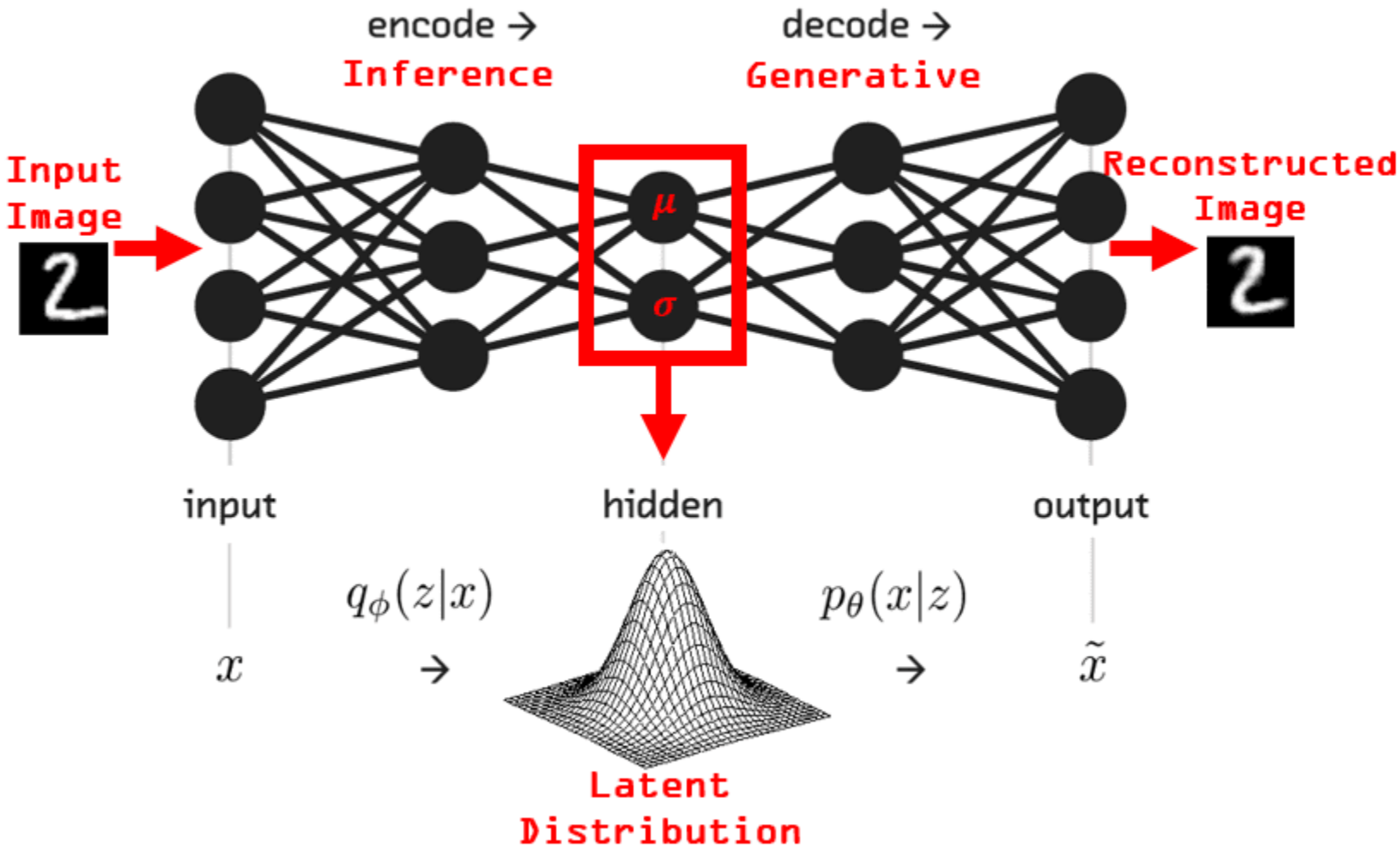
$$q(z|x) = \mathcal{N}(\mu(x), \sigma^2(x))$$

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$






Input image
Encoder
Distribution over latent space defined by z_mean and z_log_var
Point randomly sampled from the distribution
Decoder
Reconstructed image


encode →
Inference
decode →
Generative
Input
Image
Reconstructed
Image
μ
σ
input
hidden
output
$q_\phi(z|x)$
$p_\theta(x|z)$
x
$\tilde{x}$
Latent
Distribution

Sampling a point from a normal distribution

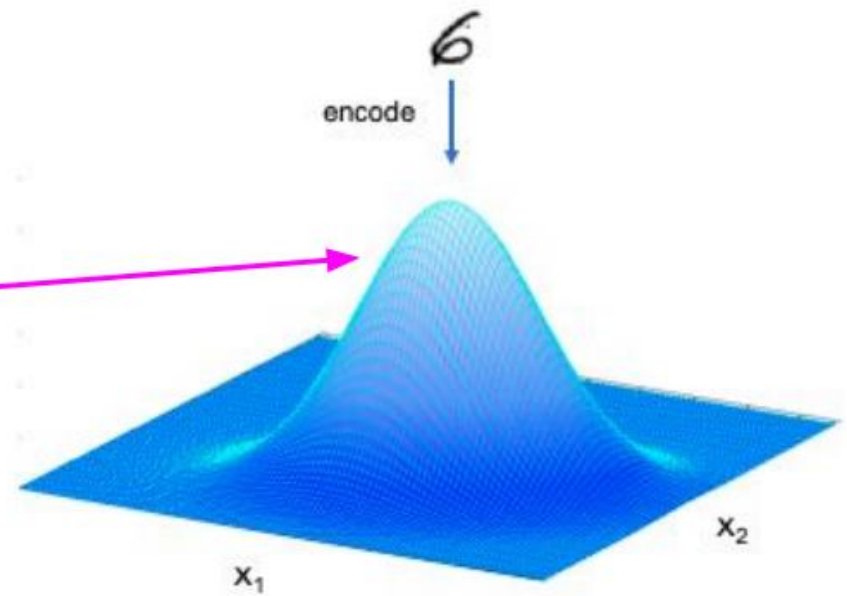
$$z = \mu + \sigma\varepsilon$$

Sampled point from
standard normal distribution

VAE and multivariate normal distribution

- VAE assumes all dimensions are independent ($\rho = 0$)
- Encoder maps each input to a mean vector and a variance vector in the latent space

$$\vec{\mu} = \begin{pmatrix} \mu_x \\ \mu_y \end{pmatrix} \quad \vec{\sigma}^2 = \begin{pmatrix} \sigma_x^2 \\ \sigma_y^2 \end{pmatrix}$$



Why We Need a Probability Distribution for z

Because the model must be **generative**, it must allow **sampling** new latent codes z to generate new data.

So instead of assigning one deterministic latent code per data point,
we want **a probability distribution** $q_\phi(z|x)$:

> "Given input x, what's the probability that the latent variable takes different values?"

That's what the **encoder** outputs.

Loss function: Variational Autoencoder

$$LOSS = RMSE$$

???

The Probabilistic View of VAE

A **VAE** is a **generative probabilistic model** — it assumes that every observed data point x (e.g., an image, speech frame, etc.) is generated from some **latent variable** z that we cannot observe directly.

We assume a **latent variable model**:

$$p_\theta(x, z) = p_\theta(x|z)\, p(z)$$

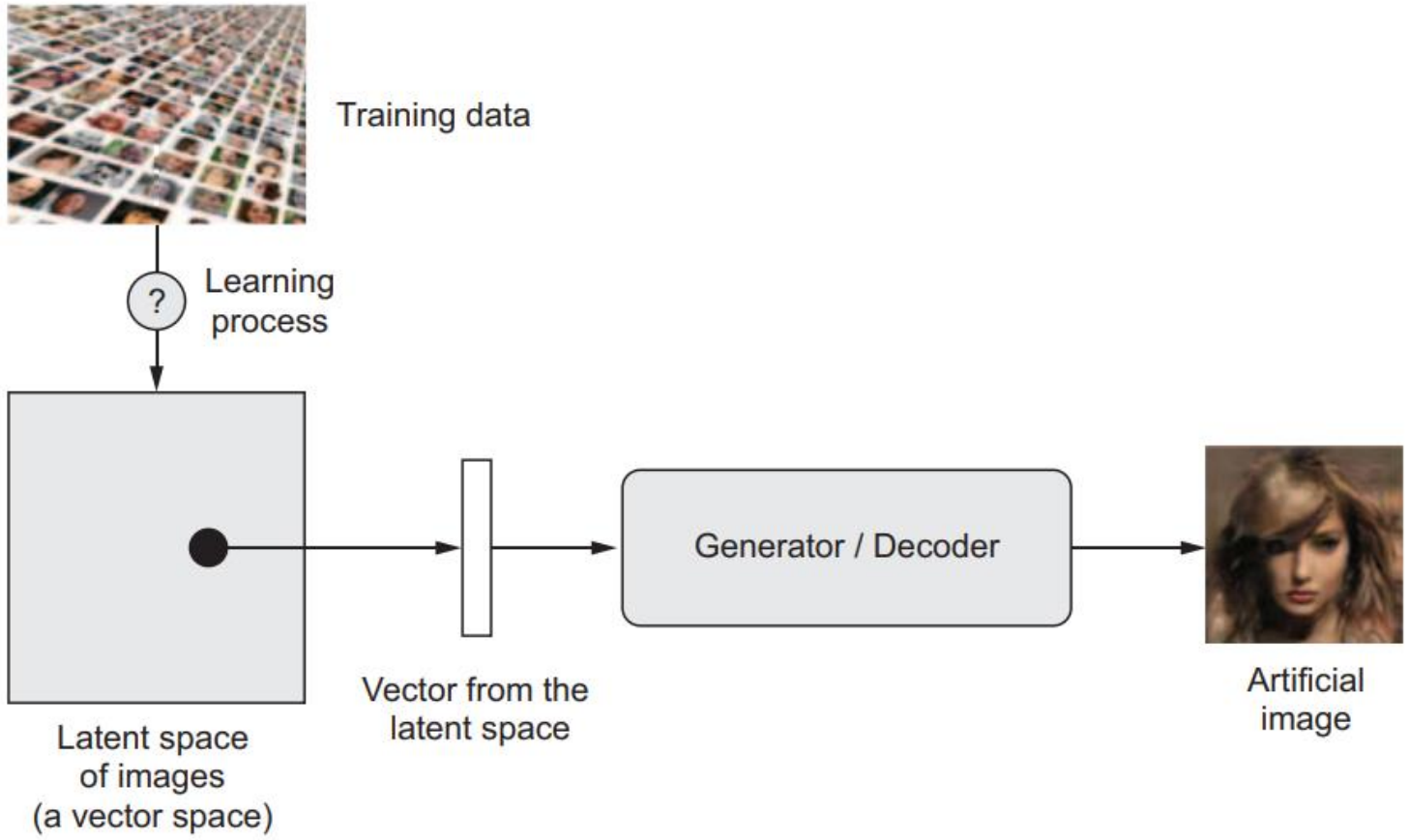
- $p(z)$: prior distribution over latent space
  → usually $\mathcal{N}(0, I)$ (a standard Gaussian).
- $p_\theta(x|z)$: **likelihood model** — defines how latent variable z generates observed data x.
  This is implemented by the **decoder network**, parameterized by θ.

Hence, to **generate** a new sample:

1. Sample $z \sim p(z)$
2. Sample $x \sim p_\theta(x|z)$

Deep Generative Learning

Image generation with deep learning is done by learning latent spaces that capture statistical information about a dataset of images. By sampling and decoding points from the latent space, you can generate never-before-seen images.



A major autoencoder drawback

There are discontinuities
in the latent space

Some generated images
will be poor

Connection to Feynman's Idea

What I cannot create, I do not understand."

- AE **copies** data (surface-level understanding).
- VAE **creates** new data by understanding the **underlying distribution** — the *true generative mechanism* of the data.

Thus, **VAE = understanding by creation** — in the spirit of Feynman's quote.

Generative Adversarial Networks (GANs)

In a **VAE**, we learned to model data probability $p(x)$ explicitly by learning $p(x|z)$ and $q(z|x)$.
In **GAN**, we skip modeling this probability directly — instead, we **train two networks to compete** so that one learns to *generate realistic data*.

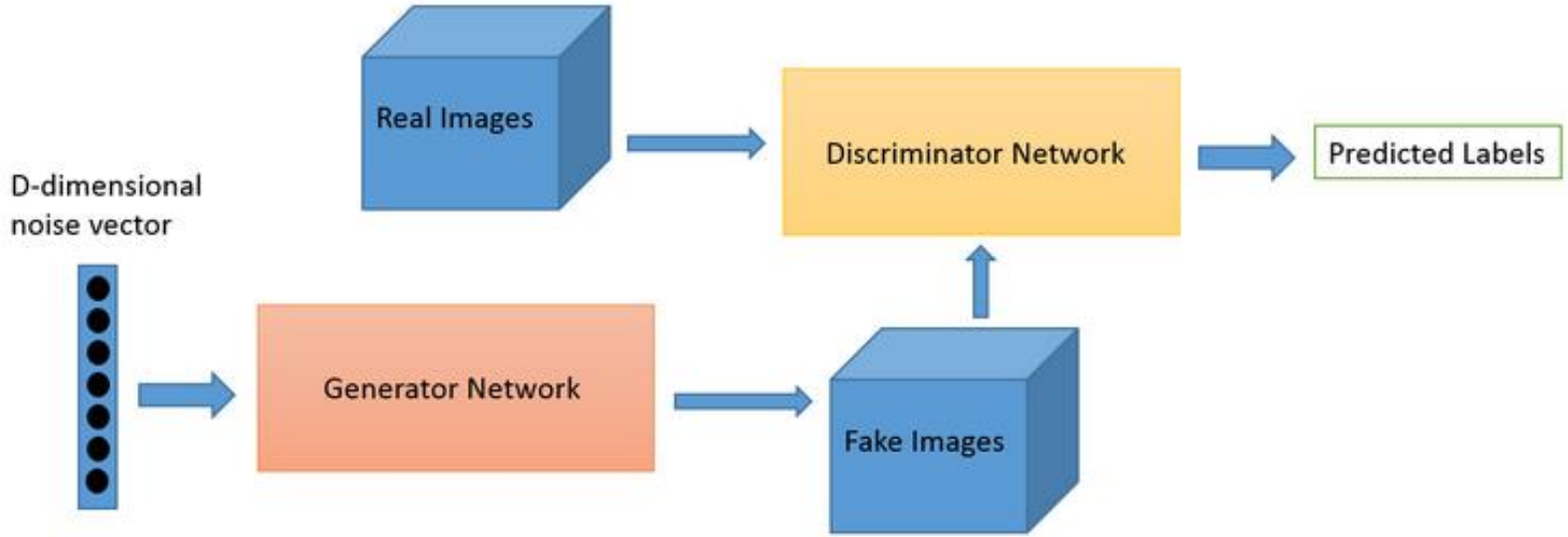
- VAE is a **probabilistic approach** (learns explicit distribution).
- GAN is a **game-theoretic approach** (learns implicitly by competition).

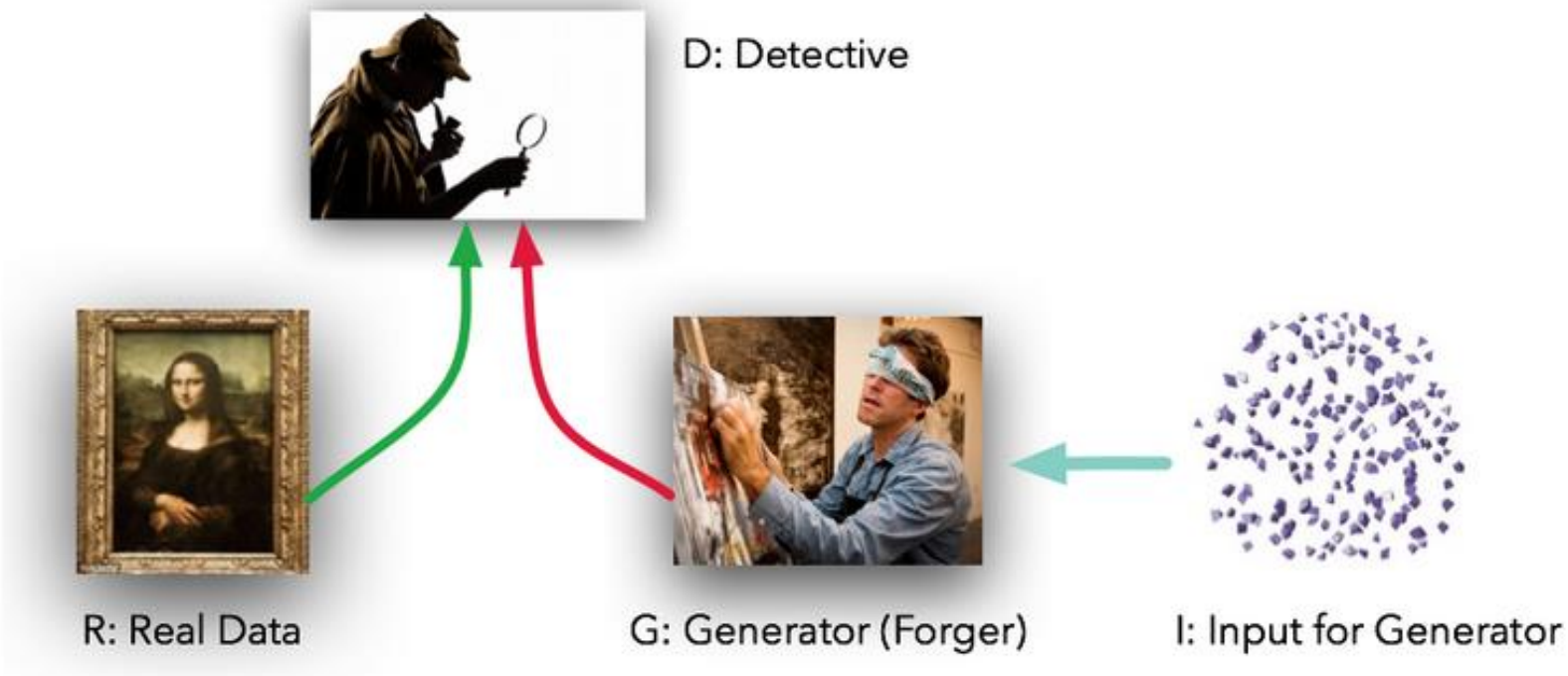
Generative Adversarial Networks (GANs)

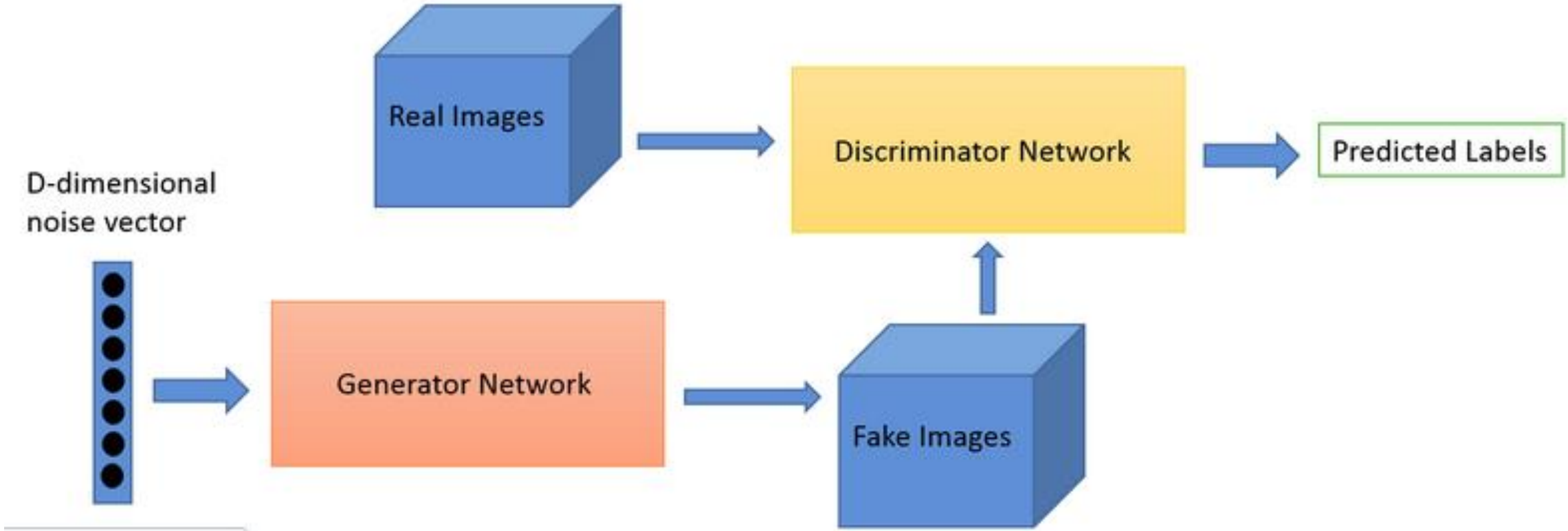
Core idea:

> Two neural networks — *Generator* and *Discriminator* — play a game.

- **Generator (G):** takes random noise → generates fake samples (like a forger).
- **Discriminator (D):** takes real or fake → decides if it's real (like a detective).
- Both improve together: G learns to fool D; D learns to detect fakes.

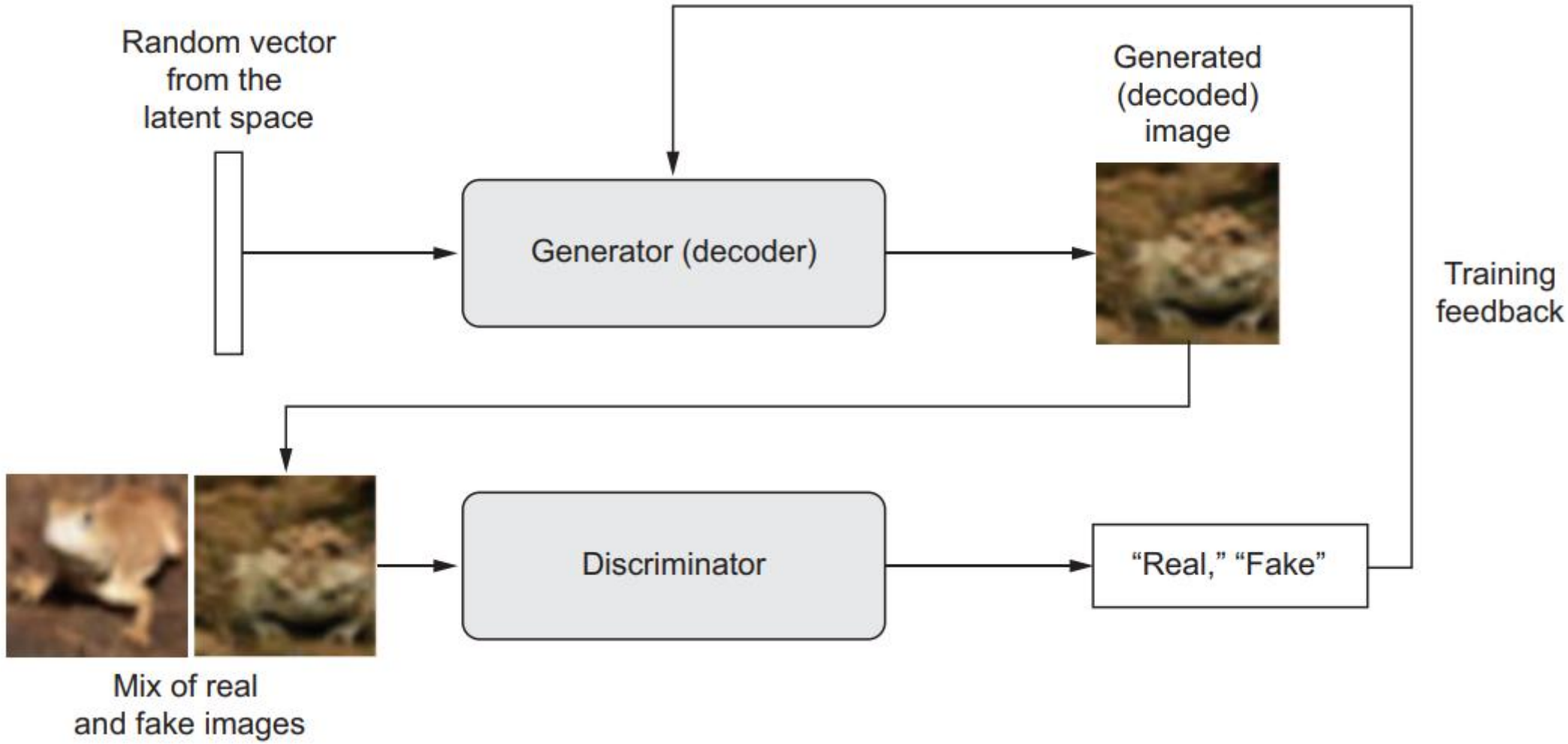

Real Images
Discriminator Network
Predicted Labels
D-dimensional
noise vector
Generator Network
Fake Images


D: Detective
R: Real Data
G: Generator (Forger)
I: Input for Generator


Real Images
Discriminator Network
Predicted Labels
D-dimensional
noise vector
Generator Network
Fake Images

There are **two neural networks**:

Network	Symbol	Role
Generator	$G(z;\theta_G)$	Takes random noise $z \sim p_z(z)$ and generates fake data $x_{fake} = G(z)$
Discriminator	$D(x;\theta_D)$	Takes an image (real or fake) and outputs a probability $D(x) \in [0,1]$ that it is real


Random vector
from the
latent space
Generator (decoder)
Generated
(decoded)
image
Training
feedback
Discriminator
“Real,” “Fake”
Mix of real
and fake images

Adversarial Training → core heart of GANs

"Adversarial" simply means two models are trained in opposition — each one's success creates difficulty for the other.

In a **GAN**,

- The **Generator** (**G**) tries to **fool** the Discriminator (D).
- The **Discriminator** (**D**) tries to **detect** that G's outputs are fake.

The Two Competing Objectives

Discriminator's Objective

- D takes an input (either real or generated).
- Outputs a probability $D(x) \in [0, 1]$:
  - $D(x) = 1$ → likely real
  - $D(x) = 0$ → likely fake

Discriminator wants to **maximize**:

$$\mathbb{E}_{x \sim p_{data}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))$$

It should classify **real as real** and **fake as fake**.

Generator's Objective

Generator creates fake data $G(z)$ from random noise $z \sim p_z(z)$.

It wants to **fool D**, i.e., make $D(G(z))$ as close to 1 as possible.

So G tries to **minimize**:

$$\mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))]$$

The Adversarial (Minimax) Game

The combined objective is:

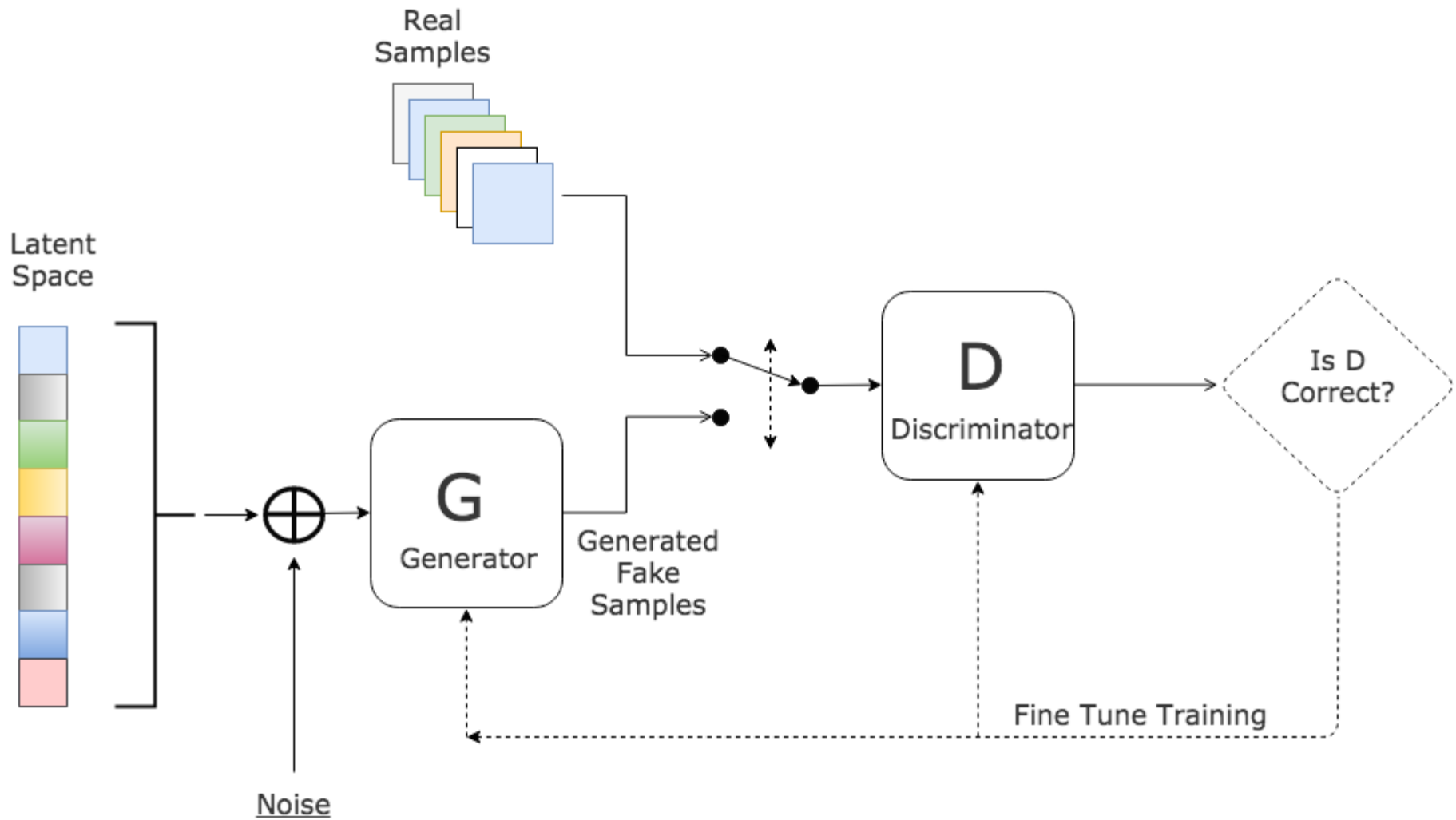
$$\min_G \max_D V(D,G) = \mathbb{E}_{x \sim p_{data}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))]$$

- D wants to **maximize** $V(D,G)$
- G wants to **minimize** $V(D,G)$

This forms a **minimax game** — like chess or poker between two intelligent players.

Can you derive the Loss Function of Discriminator from BCE Loss?

Generative Adversarial Network

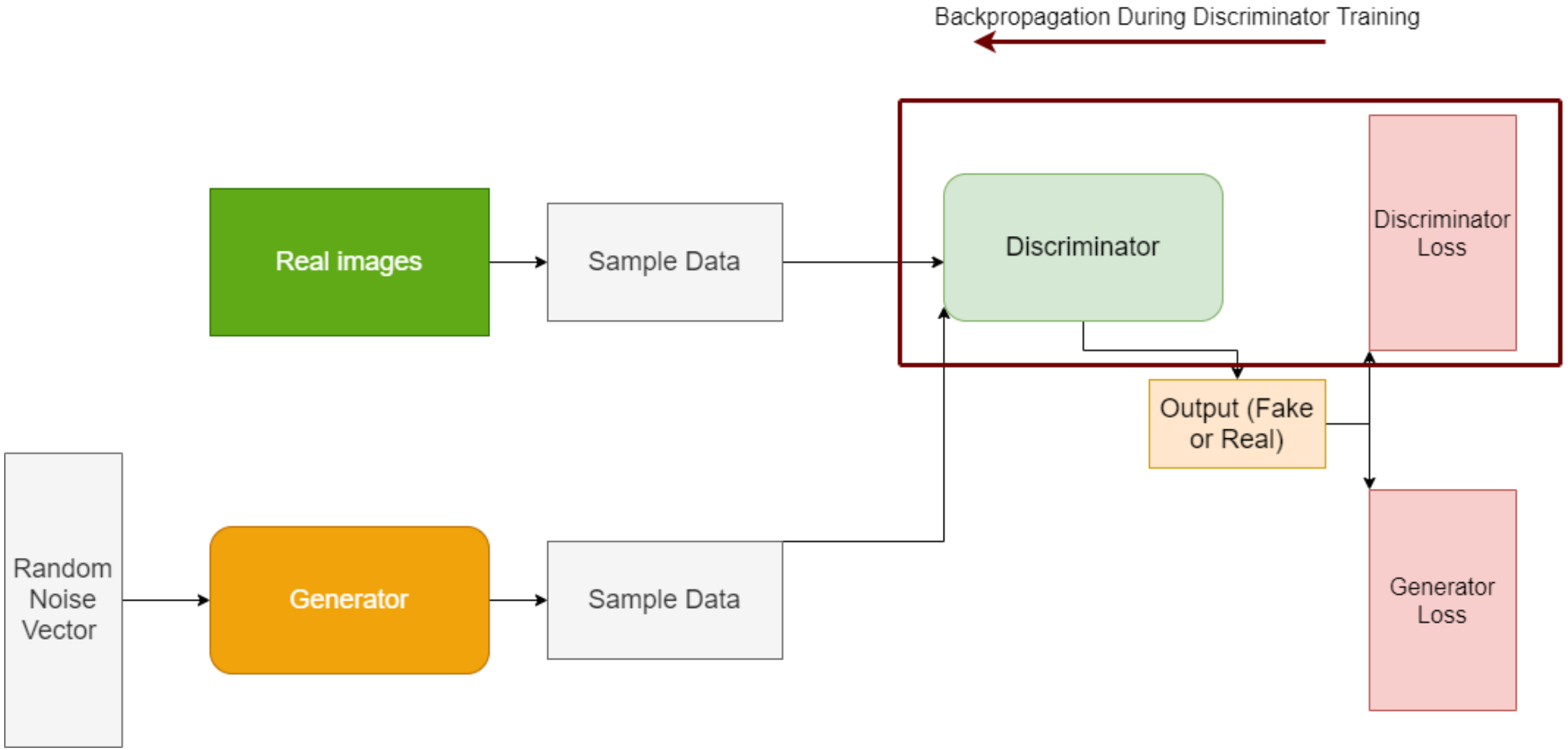


Step-by-step procedure:

1. **Train the Discriminator (D):**
   - Get a batch of real samples $x_{real} \sim p_{data}$.
   - Generate fake samples $x_{fake} = G(z)$ using noise $z \sim p_z(z)$.
   - Compute the discriminator loss:

$$L_D = -[\log D(x_{real}) + \log(1 - D(x_{fake}))]$$

   - Update D's parameters (gradient ascent).


Backpropagation During Discriminator Training
Real images
Sample Data
Discriminator
Discriminator Loss
Output (Fake or Real)
Random Noise Vector
Generator
Sample Data
Generator Loss

2. **Train the Generator (G):**
    - Sample new noise $z \sim p_z(z)$.
    - Generate fake samples $x_{fake} = G(z)$.
    - Compute generator loss (non-saturating version):

$$L_G = -\log D(G(z))$$

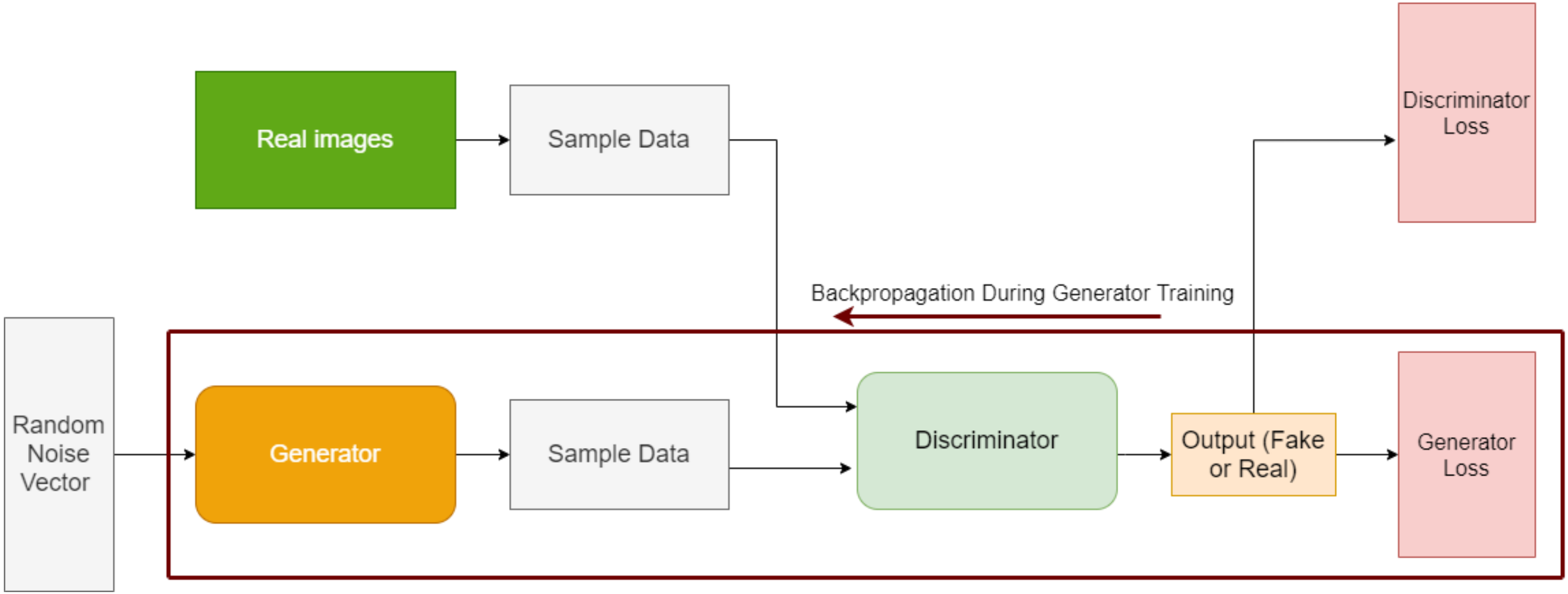
    - Update G's parameters (gradient descent) — note that G's gradients come *through* D.

3. **Alternate updates** — typically one D update per G update.

   Over time:

- D becomes a better judge.
- G becomes a better faker.

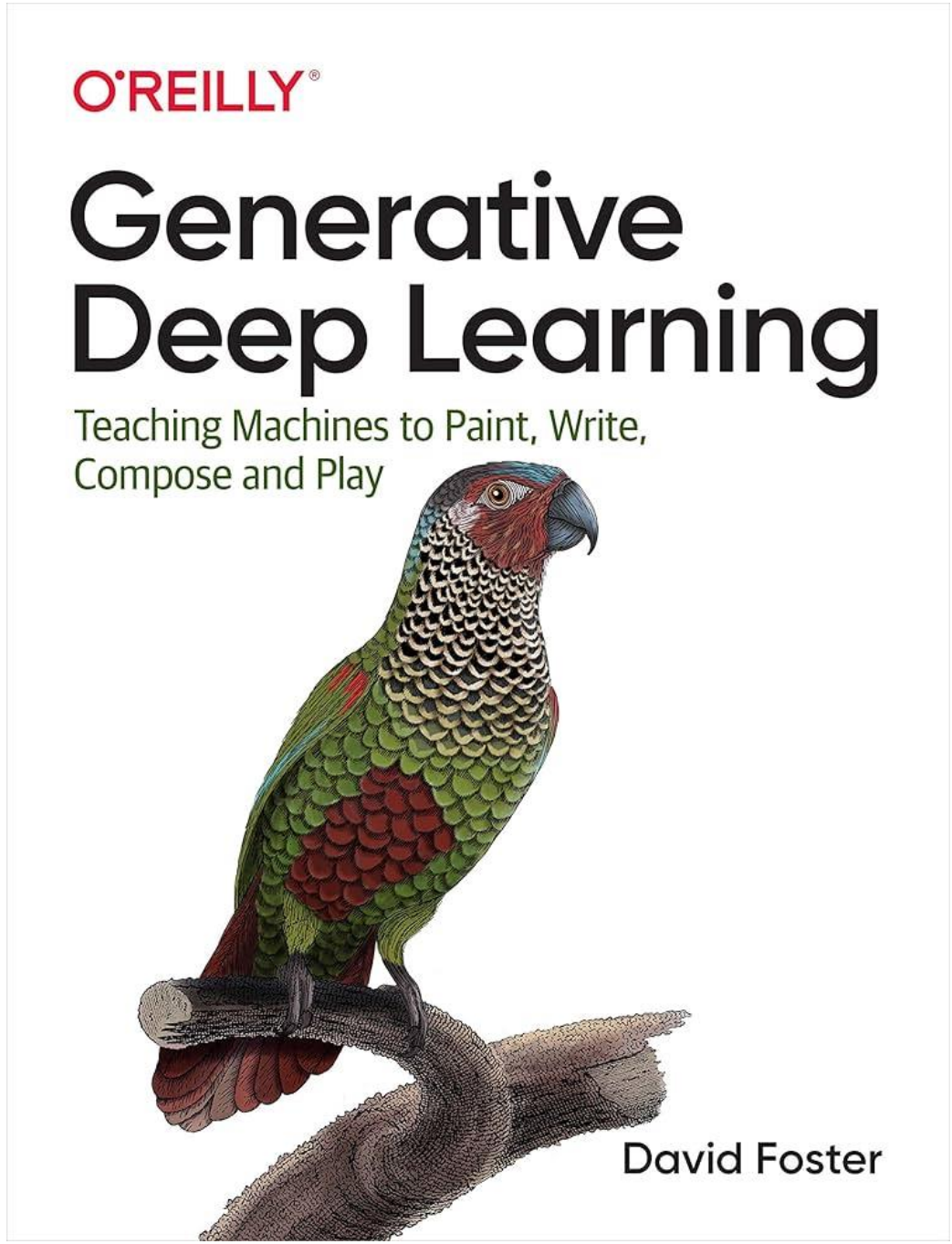
The training continues until the generated data distribution $p_g(x)$ approximates the real data distribution $p_{data}(x)$.

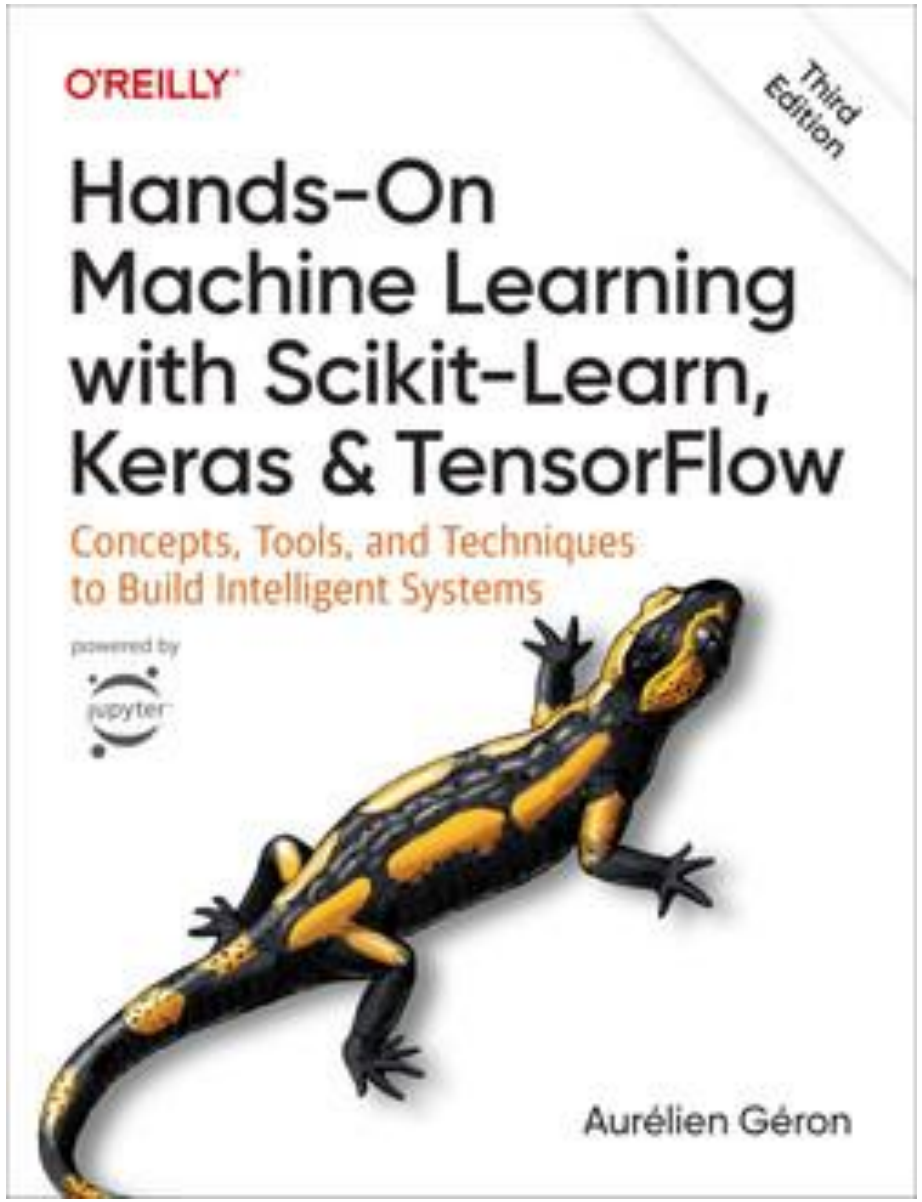

Real images
Sample Data
Discriminator Loss
Backpropagation During Generator Training
Random Noise Vector
Generator
Sample Data
Discriminator
Output (Fake or Real)
Generator Loss

GAN

https://thispersondoesnotexist.com/

A website that shows faces generated by a recent GAN architecture called StyleGAN


O'REILLY®
Generative Deep Learning
Teaching Machines to Paint, Write, Compose and Play
David Foster


O'REILLY
Third Edition
Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow
Concepts, Tools, and Techniques to Build Intelligent Systems
powered by
jupyter
Aurélien Géron

Summary

- **AE:** Learns deterministic mapping; reconstructs input from compressed latent vector.
- **VAE:** Learns probabilistic latent space; samples from $\mathcal{N}(0, I)$ to generate data.
- **GAN:** Uses two networks (Generator vs Discriminator) in adversarial setup for realistic data.
- **AE/VAE:** Optimize reconstruction-based loss; **GAN:** optimizes adversarial loss.